Введение в веб-технологии для АЗИС часть 2

Введение в PHP

В дополнение к уже изученным HTML, CSS и JavaScript, настало время познакомиться с ещё одной важной веб-технологией — языком **PHP**.

PHP — это серверный, скриптовый, интерпретируемый язык программирования, предназначенный для создания динамически формируемых веб-страниц. В отличие от JavaScript, который выполняется в браузере пользователя (на стороне клиента), PHP исполняется на сервере до того, как страница будет отправлена пользователю.

Зачем нужен PHP?

Большинство HTML-страниц являются **статичными**, то есть содержат неизменяемый контент. Такие страницы каждый раз отображаются в браузере одинаково, вне зависимости от времени суток, данных пользователя или состояния внешнего мира.

Однако современные веб-приложения, как правило, **динамичны** — они должны предоставлять информацию, которая постоянно изменяется. Примеры таких данных включают:

- курсы валют,
- прогноз погоды,
- расписания транспорта,
- персонализированные приветствия и рекомендации.

С помощью одного лишь HTML решить подобные задачи невозможно. Именно здесь на помощь приходит PHP — язык, специально созданный для **генерации HTML-кода на лету**, в момент обработки серверного запроса.

Как работает PHP?

Процесс работы PHP можно описать следующим образом:

1. Пользователь отправляет запрос к веб-серверу, например, при переходе по адресу `https://example.com/weather.php`.
2. Веб-сервер обнаруживает, что запрашиваемый файл имеет расширение `.php`, и передаёт управление **интерпретатору PHP**.
3. PHP-интерпретатор исполняет сценарий — то есть обрабатывает команды, содержащиеся в файле, и генерирует итоговый HTML.
4. Сформированный HTML передаётся обратно веб-серверу.
5. Веб-сервер отправляет HTML в браузер пользователя, где он отображается как обычная страница.

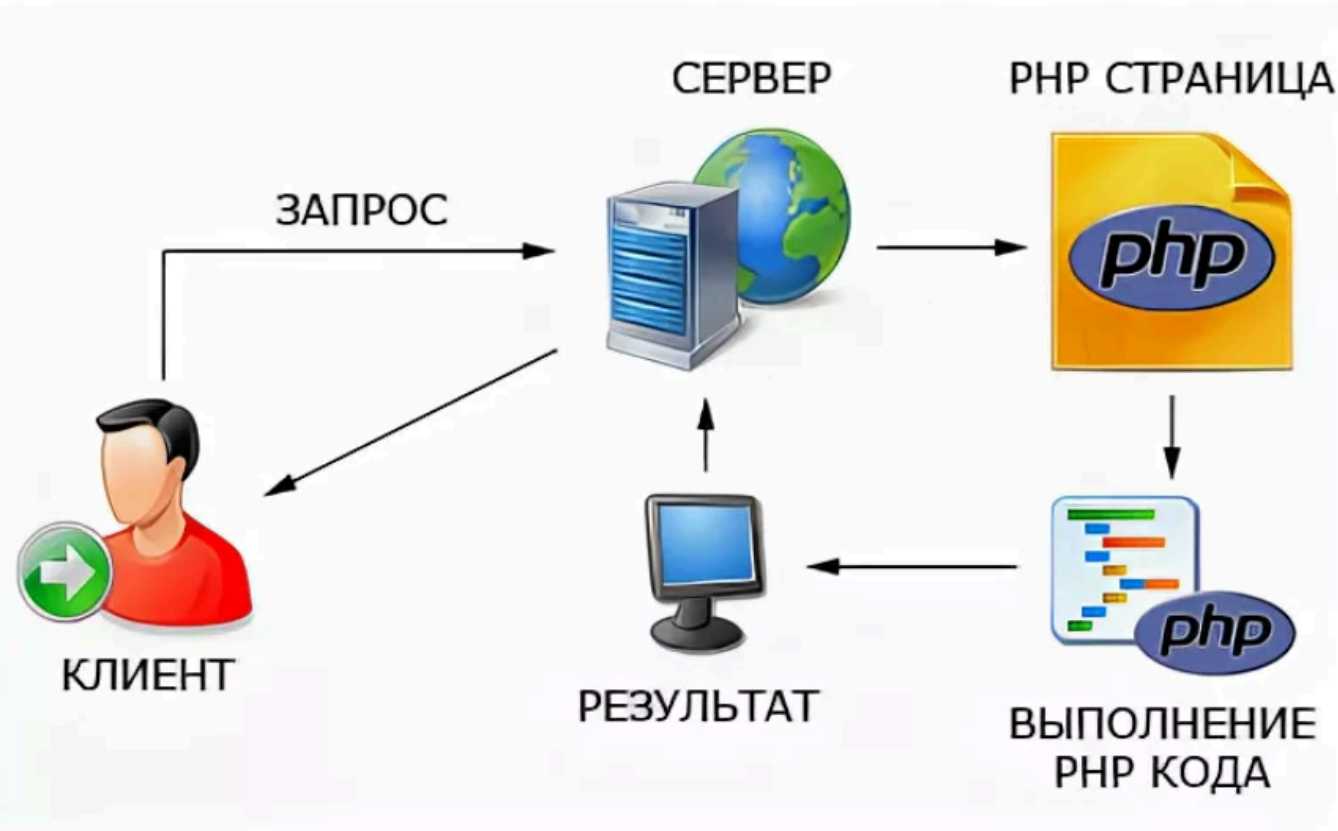


Рис. 32. Схема работы PHP

Главное достоинство PHP в том, что он позволяет изменять веб-страницу на сервере прямо перед тем, как она будет отправлена браузеру клиента.

Когда браузер запрашивает страницу с адресом, который кончается на `.php`, веб-сервер обращается к PHP и просит выполнить сценарий из файла по этому адресу.

Выполнение сценария также называют его **интерпретацией**, а сам PHP — **интерпретатором**.

Сценарий — это программа, которая находится на стороне сервера и запускается в ответ на запрос от браузера.

Настройка рабочего окружения

Для того, чтобы использовать язык php на вашем компьютере, нужно скачать и настроить локальный веб-сервер.
Так как этот курс предназначен для начинающих, то мы не будем глубоко погружаться в установку и настройку рабочего окружения, а вместо этого посоветуем вам простой кроссплатформенный инструмент, который уже содержит все, что нужно для разработки.
Перейдите на этот сайт и скачайте установщик XAMPP для своей операционной системы.
После загрузки и установки на вашем компьютере появятся свежая версия PHP, веб-сервер, база данных, и много других программ и инструментов для удобной работы.

Установка XAMPP очень проста. Достаточно следовать рекомендациям мастера установки.

После установки запустите XAMPP Control Panel. Вы увидете подобное окно (Рис.33.)

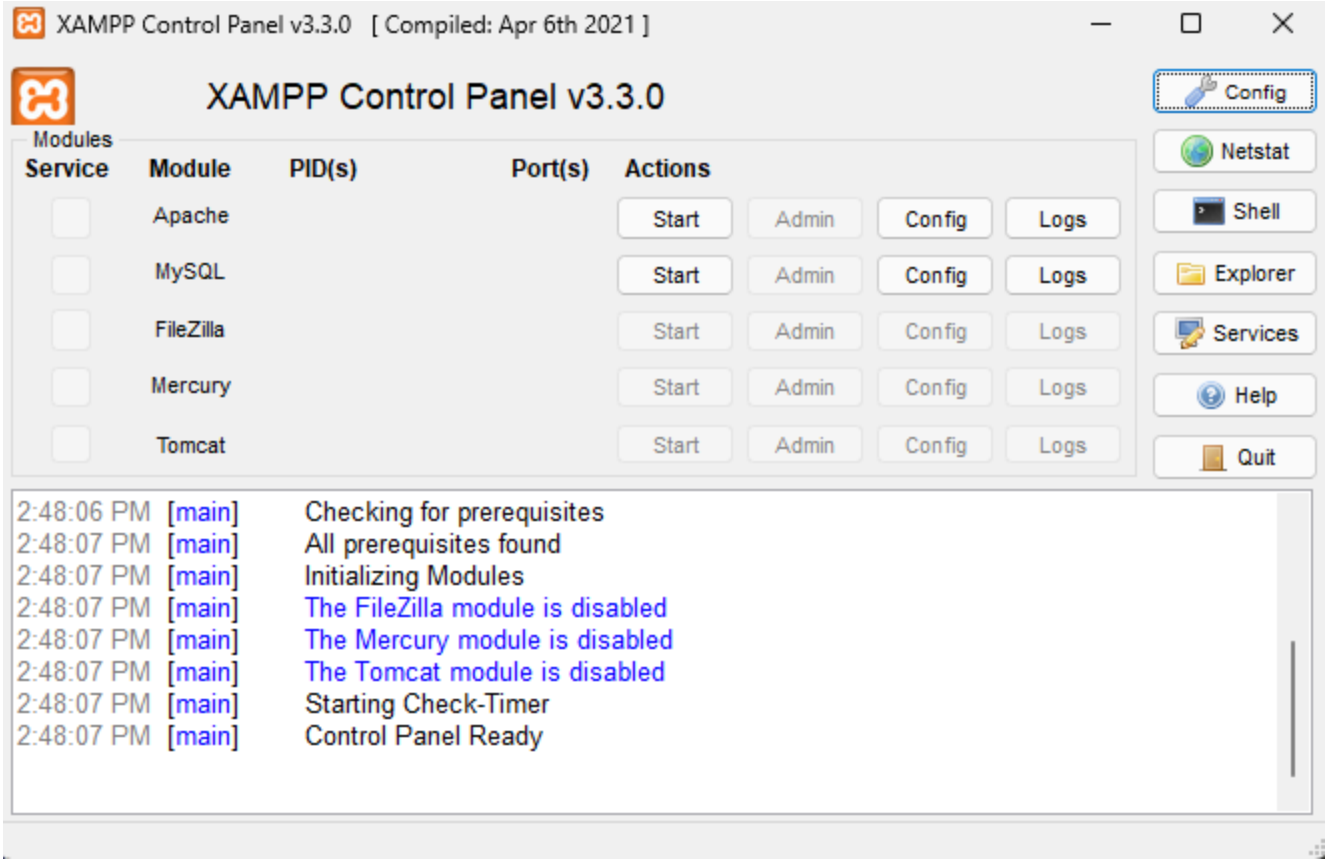


Рис.33 Окно XAMPP Control Panel

Далее необходимо перенести файлы сайта, который мы создавали в директорию, из которой их будет обрабатывать локальный веб-сервер Apache. Кроме того, необходимо будет изменить расширение наших html файлов.
Для этого:

- перейдите на своем компьютере в место установки XAMPP (папка xampp. Для быстрого перехода туда нажмите кнопку "Explorer" в окне XAMPP Control Panel)
- в папке xampp найдите директорию htdocs и перейдите в нее
- создайте внутри htdocs директорию BMSTU
- перенесите туда все файлы сайта вместе с папкой img и ее содержимым
- переименуйте файлы index.html и contacts.html в index.php и contacts.php
- не забудьте поменять расширения страниц в навигации сайта (замените .html на .php)

После этого включите локальный веб-сервер Apache. Для этого нажмите кнопку "Start" напротив надписи Apache на XAMPP Control Panel. (Рис.34)

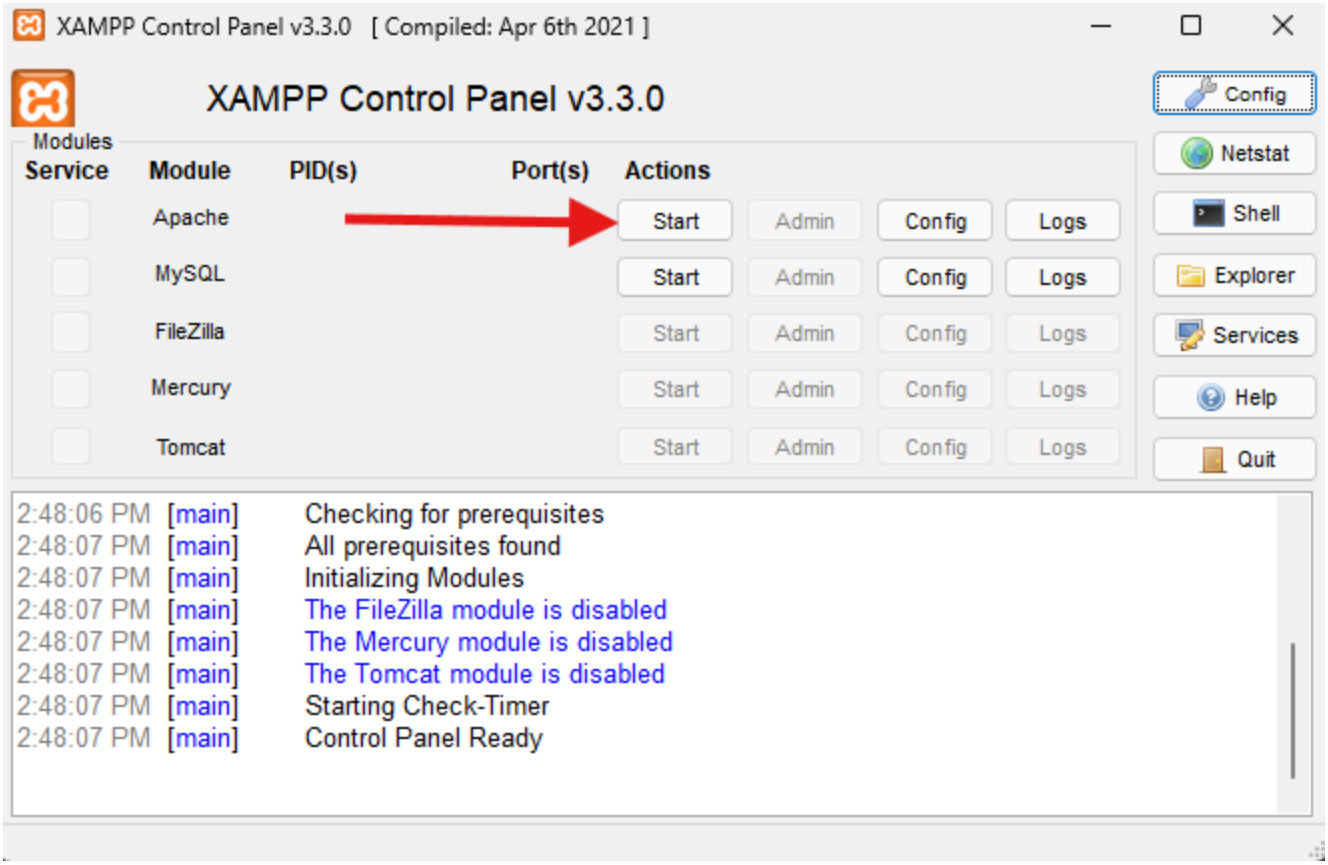


Рис.34. Запуск локального веб-сервера Apache через XAMPP Control Panel.

Откройте браузер и в адресной строке введите:

```
http://127.0.0.1/BMSTU/
```

Вы должны увидеть главную страницу нашего сайта.

Основы синтаксиса языка PHP

Первый сценарий на PHP

PHP-сценарий представляет собой последовательность выражений, которые интерпретатор выполняет сверху вниз. Каждое выражение, как и в JavaScript, завершается точкой с запятой.

Каждый PHP-сценарий обязательно начинается с такой строки: `<?php` — так мы сообщаем веб-серверу, что дальше в файле находится код на языке PHP.

Каждый PHP-сценарий заканчивается такой строкой: `?>` - так мы сообщаем серверу, где заканчивается наш PHP-сценарий. В случае если наш код последний в документе, и после него ничего не следует, этот фрагмент не обязателен.

Наш первый сценарий будет состоять всего из одной инструкции, которую мы разместим в "Подвале" нашего сайта.

```
<footer>
   <?php print("МГТУ им. Н.Э. Баумана © Все права защищены"); ?>
</footer>
```

Переменные в PHP

Переменные в PHP — это контейнеры для хранения данных. Вы можете думать о них как о ящиках, на которых есть имена.
В синтаксисе PHP имя переменной записывается латинскими символами, но первым символом всегда должен быть знак доллара `$`, а затем идёт имя.
Знак «равно» в PHP является *оператором* и означает операцию присваивания.

```
$name = "Николай";
$age = 42;
```

Обратите внимание, что также как и в языке javascript, с которым вы уже познакомились, в языке php существует тип данных строка. И точно так же, при присваивании текстовой строки в переменную, необходимо брать строку в кавычки.

Код выше означает:

- поместить строку "Николай" в переменную $name
- поместить число 42 в переменную $age

Очевидно, что впоследствии к переменным необходимо обращаться по их имени.
Перепишем наш первый сценарий, чтобы это продемонстрировать:

```
<footer>
   <?php
       $footer_text = "МГТУ им. Н.Э. Баумана © Все права защищены";
       print($footer_text);
   ?>
</footer>
```

Конкатенация в PHP

Напомним, что этим страшным словом называется операция склеивания нескольких строк в одну. Для конкатенации в PHP оператор конкатенации — `.` (точка)
Еще раз модифицируем наш "подвал" для демонстрации.

```
<footer>
   <?php
       $footer_text_start = "МГТУ им. Н.Э. Баумана ";
       $footer_text_end = "© Все права защищены";
       $united_string = $footer_text_start . $footer_text_end;
```

```
        print($united_string);
    ?>
</footer>
```

Операторы в PHP

Оператором в программировании называется команда, обозначающая определенное математическое или логическое действие, выполняемое с данными (операндами)

PHP дополняет уже знакомые нам из арифметики операторы, несколькими новыми:

- уже знакомый оператор присваивания `=`, который используется для присваивания значения переменной;
- очень полезны операторы сравнения: `==`, `≠`, `>`, `<` — равно, не равно, больше и меньше;
- `%` — оператор остатка от деления одного числа на другое.

```
print(5 % 2); // Выведет: 1
```

Комментарии в PHP

PHP поддерживает три способа добавления комментариев:

```
<?php
echo "Это тест"; // Это однострочный комментарий в стиле C++
/* Это многострочный комментарий
ещё одна строка комментария */
echo "Это ещё один тест";
echo "Последний тест"; # Это комментарий в стиле оболочки Unix
?>
```

Условные конструкции в PHP

Когда мы пишем программу, нам необходимо уметь выполнять различные действия в зависимости от условия.
Для этого используется оператор `if`.
Например, мы хотим показывать пользователям онлайн-книотеатра отзывы на фильмы по их рейтингу. Мы спросили рейтинг фильма у пользователя, сохранили его в переменную, а теперь хотим вывести различные отзывы в зависимости от рейтинга. Здесь пригодятся операторы `if` и `else`.

```
<?php
if ($rating > 8) {
    print("Отличный фильм");
} else {
```

```
    print("Нормальный или плохой фильм");
}
```

Из примера видно, что условная конструкция состоит из трёх частей:

- условие `if` — выражение, которое возвращает истинный или ложный результат;
- блок `if` — строки кода, которые будут выполнены, если условие вернуло истину;
- блок `else` — строки кода, которые будут выполнены, если условие вернуло ложь.

Обратите внимание, что каждый блок обрамляется и ограничивается фигурными скобками.

Пример использования PHP

Как известно, 13 июля 1830 года был основан МГТУ им. Н.Э. Баумана.
Предлагается с помощью языка PHP вычислить количество дней прошедших с этой памятной даты до сегодняшнего дня и вывести эту информацию нам на сайт.

```
$now = time(); // текущее время (метка времени)

$your_date = strtotime("1830-07-13"); // дата основания МГТУ в строке

$datediff = $now - $your_date; // получим разность дат (в секундах)

$days = floor($datediff / (60 * 60 * 24)); // вычислим количество дней из разности дат
print("Создаем инженеров уже " . $days . " дня/дней" . "<br>");
```

Давайте разберем этот код по шагам:

1. `$now = time();` функция возвращает текущее время в виде метки времени (`timestamp`) - количество секунд, прошедших с начала эпохи Unix (1 января 1970 года).
   Данное значение присваивается переменной $now.
2. `$your_date = strtotime("1830-07-13");` функция преобразует строку `"1830-07-13"` в метку времени Unix. Данное значение присваивается переменной `$your_date`.
3. `$datediff = $now - $your_date;` вычисляет разницу между текущим временем и временем, соответствующим дате "1830-07-01". Результат будет представлять собой разницу в секундах между этими двумя метками времени. Данное значение присваивается переменной `$datediff` .
4. `floor($datediff / (60 * 60 * 24));` делит разницу в секундах на количество секунд в одном дне (60 секунд *60 минут* 24 часа) и округляет результат в меньшую сторону с помощью функции `floor()`. Таким образом, мы получаем количество дней, прошедших между текущей датой и датой основания МГТУ.
5. Записываем количество прошедших дней в переменную `$days`

6. Печатаем результирующую строку на экран.

Обратите внимание на `<br>`, в конце строки. Это специальный HTML тэг делающий перевод строки в тексте.
Текст после `<br>` начинается со следующей строки текстового блока, делающую перевод строки на одну вниз.

Массивы PHP

Познакомимся с еще одним типом данных в php под названием массив. Главное отличие массива от остальных типов данных заключается в его способности хранить в переменной больше одного значения.

В предыдущих примерах имя переменной было ассоциировано только с одним значением:

```
$name = "Николай"
$age = 42;
```

С помощью массива можно хранить в переменной несколько значений сразу:

```
$bmstu_buildings = ["ГУК", "УЛК", "СМ", "Э"];
```

В этом примере мы сохранили в переменной $bmstu_buildings сразу четыре значения аббревиатур названий корпусов МГТУ.
Но сохранить эти данные — это только половина дела. Как с ними потом работать?
Уже знакомый вам способ вывода переменной на экран не будет работать с массивами:

```
<?php
    print("Аббревиатуры корпусов МГТУ: " . $bmstu_buildings);
?>
```

Дело в том, что массив — это не обычная переменная. Для работы с массивами нужен особый подход. Внутри массива у каждого значения есть индекс. Индекс - это просто порядковый номер значения внутри массива.

Индексация начинается с нуля, так что первый элемент получает индекс — `0`, второй — `1`, и так далее.

Чтобы получить определенный элемент массива, необходимо знать его индекс.
Напечатаем названия всех сериалов из массива через запятую:

```
<?php
    print("Аббревиатуры корпусов МГТУ: " . $bmstu_buildings[0] . ", " .
$bmstu_buildings[1] . ", " . $bmstu_buildings[2] . ", " .
```

```
$bmstu_buildings[2]);
?>
```

Массивы позволяют перезаписывать существующие значения и добавлять новые. Добавить новое значение в массив можно так:

```
$bmstu_buildings[] = "ДТ (Дворец технологий)";
```

Новый элемент автоматически получит индекс равный максимальному индексу из существующих + 1. «ДТ (Дворец технологий)» сохранится в массиве под индексом 4.
Если нам необходимо, значения в массиве можно заменить. Чтобы вычеркнуть старое значение и заменить его новым, нужно присвоить новое значение любому из существующих в массиве индексов:

```
$bmstu_buildings[1] = "УЛК / БЗДК";
```

Для удаления элемента массива значения по его индексу существует функция `unset`:

```
unset($bmstu_buildings[2]);
```

Ассоциативные массивы

Мы уже познакомились с простыми массивами, но для понимания всего материала курса нам нужно двинуться чуть дальше - в сторону еще одного типа данных под названием ассоциативные массивы.

Ассоциативные массивы отличаются от простых тем, что вместо индексов у них ключи. Если индекс всегда является целым, порядковым числом, то ключ может быть любой произвольной строкой.

Мы уже знаем многое об МГТУ: полное название,год основания, аббревиатуры корпусов. Мы умеем работать с переменными и можем сдедать так, чтобы то, что мы знаем об МГТУ находилось в разных переменных.Но это не удобно. Гораздо лучшей идеей было бы хранить все эти данные в одном месте, и именно в таких ситуациях помогают ассоциативные массивы.
Приведем пример:

```
<?php
$bmstu = ['foundation_year' => 1830, 'full_name' => 'МГТУ им. Н.Э. Баумана',
buildings => ["ГУК", "УЛК", "СМ", "Э"] ];
?>
```

Обратите внимание: массив может содержать другой массив в качестве одного из значений. В нашем примере мы поместили простой массив внутри ассоциативного под ключом `buildings`.
Вывод информации из ассоциативных массивов аналогичен простым массивам, только вместо индексов мы передаем ключи:

```
<?php
print($bmstu['full_name']);
?>
```

Формы и PHP

В предыдущих разделах курса мы уже познакомились с HTML-формами. Напомним, что форма — это структурный элемент HTML-документа, предназначенный для ввода и передачи данных от пользователя на сервер. Формы играют ключевую роль во взаимодействии клиента с веб-приложением: они используются при регистрации, авторизации, оформлении заказов, отправке сообщений и во многих других сценариях.

С помощью формы можно передавать как текстовую информацию (например, имя и сообщение), так и бинарные данные — в частности, изображения и другие файлы.

HTML определяет структуру формы: какие поля она содержит, в каком порядке они расположены. CSS отвечает за внешний вид этих элементов. Однако сами по себе HTML и CSS не обеспечивают обработку пользовательских данных. Для этого необходим сервер, принимающий запрос, и скрипт на стороне сервера — например, на языке PHP, — который обрабатывает переданную информацию и формирует ответ.

Ранее мы уже взаимодействовали с формами с помощью JavaScript: обрабатывали данные прямо в браузере, не передавая их на сервер. Теперь же мы перейдём к полноценной серверной обработке — с использованием языка PHP.

> Хорошее понимание происходящего — залог успешного обучения. Поэтому давайте разберёмся, что именно будет происходить «под капотом».

Мы создадим HTML-форму, которая отправляет данные на локальный веб-сервер через HTTP-запрос. Сервер примет запрос, передаст его содержимое обработчику, написанному на PHP, и в ответ сформирует веб-страницу с результатами обработки. Эта страница будет возвращена клиенту в теле HTTP-ответа и отображена в браузере.

Перед тем как приступить, необходимо подготовить окружение. На текущий момент форма обрабатывается с помощью JavaScript. Мы отключим этот механизм, чтобы передать управление PHP-скрипту.

Откройте файл `contacts.php` и закомментируйте подключение скрипта `xss-test.js`. Ваш код должен выглядеть следующим образом:

```
    </footer>
```

```
    <!-- <script src="script.js"></script> -->
    <!-- <script src="xss-test.js"></script> -->
    </body>

</html>
```

Следующим шагом является корректная настройка формы для отправки данных на сервер. Для этого необходимо задать два ключевых атрибута тега `<form>`: `method` и `action`. Эти атрибуты определяют способ передачи данных и адрес скрипта, который будет их обрабатывать.

Атрибут `method` указывает HTTP-метод, который будет использоваться при отправке формы. Как вы уже знаете, наиболее часто используются два метода:

- `GET` — применяется для запроса ресурса и передачи данных в URL-строке;
- `POST` — используется для отправки пользовательских данных в теле HTTP-запроса, что делает его предпочтительным для форм, содержащих текстовые поля, файлы и конфиденциальную информацию.

Поскольку мы хотим передать на сервер пользовательские данные (возможно, включая вложения), укажем метод POST:

```
method="POST"
```

Атрибут `action` определяет путь к PHP-скрипту, который будет обрабатывать отправленные данные. В нашем случае это файл `form.php`, расположенный в том же каталоге:

```
action="form.php"
```

После добавления атрибутов, убедитесь, что параграф с классом `welcome-user` (использовавшийся ранее для JavaScript-обработки) удалён из HTML, так как теперь мы обрабатываем данные на сервере. Затем создайте новый файл `form.php` в корневой папке проекта.

Обработка формы на PHP

Как было отмечено ранее, после нажатия пользователем кнопки «Отправить» браузер отправляет HTTP-запрос на сервер. Указанный в `action` PHP-скрипт принимает этот запрос, обрабатывает переданные данные и формирует ответ.

Полученные от пользователя данные автоматически сохраняются в глобальном ассоциативном массиве `$_POST`, если форма отправлена методом `POST`. Помимо него, в PHP существуют и другие суперглобальные массивы:

- `$_GET` — используется для получения данных, переданных методом `GET`;

- `$_FILES` — содержит информацию о загружаемых пользователем файлах.

Каждое поле формы будет представлено в массиве `$_POST`, где ключом служит значение атрибута `name`, а значением — ввод пользователя.

Чтобы вывести из формы всю информацию на экран, можно в файле form.php написать такой сценарий:

```
<?php
    if (isset($_POST)) {
        print("Имя: " . $_POST['username']);
        print("<br>Сообщение: " . $_POST['message']);
    }
?>
```

Здесь функция `isset()` проверяет, были ли отправлены данные методом `POST`. Далее скрипт извлекает и отображает значения из массива `$_POST`. Результат выполнения кода показан на рисунке 35.

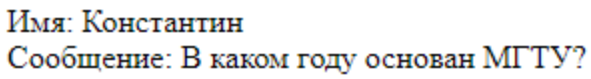


Рис. 35. Результат обработки формы с помощью языка PHP

Передача файлов через форму

Формы HTML позволяют не только отправлять текст, но и передавать файлы на сервер. Для этого необходимо:

1. Добавить в тег `<form>` атрибут `enctype="multipart/form-data"`, который указывает браузеру, что форма может содержать бинарные данные.
2. Использовать поле ввода с типом `file`.

Модифицированная версия формы будет выглядеть так:

```
 <form class="contacts__form" action="form.php" method="POST"
enctype="multipart/form-data">
            <label class="contacts__form-label" for="username">Как вас
зовут?</label>
            <input class="contacts__form-input" type="text" id="username"
name="username"><br><br>
            <label class="contacts__form-label" for="message">Сообщение
```

```
</label>
            <textarea class="contacts__form-textarea" id="usertext" rows="5"
name="message" placeholder="Расскажите обо всём, что для вас важно"
id="message"></textarea>
            <br><br>
            <label class="contacts__form-input" >Ваш аватар:
              <input type="file" name="avatar">
            </label>
            <br>
            <br>
            <input class="contacts__form-submit" type="submit"
value="Отправить">

          </form>
```

Загруженные файлы становятся доступны через массив `$_FILES`. Добавим обработку файла в скрипт `form.php`:

```
<?php

    if (isset($_POST)) {
        print("Имя: " . $_POST['username']);
        print("<br>Сообщение: " . $_POST['message']);
    }
    if (isset($_FILES['avatar'])) {
        $file = $_FILES['avatar'];
        print("<br>Загружен файл с именем " . $file['name'] . " и размером "
. $file['size'] . " байт");
    }
?>
```

В данном скрипте проверяется, был ли загружен файл, а также с помощью массива `$_FILES['avatar']` извлекается имя файла (`$file['name']`) и его размер в байтах (`$file['size']`).

Результат обработки формы, включающей загрузку файла, показан на рисунке 36.

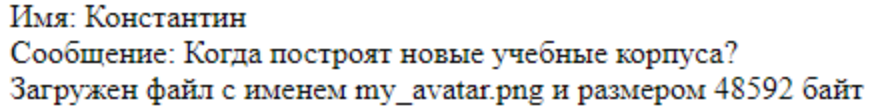


Рис.36. Результат обработки обновленной формы

Еще пара веб-уязвимостей

Информационная безопасность является неотъемлемой частью современной веб-разработки. В этом разделе мы рассмотрим две критически важные уязвимости, знание которых необходимо каждому специалисту в области веб-технологий:

- **Command Execution (удалённое выполнение команд)**
- **File Upload (неконтролируемая загрузка файлов)**

Уязвимость Command Execution

Command Execution (удалённое выполнение команд) — это одна из наиболее опасных уязвимостей, связанная с возможностью выполнения произвольных команд операционной системы на сервере, где работает веб-приложение.

Если злоумышленнику удаётся внедрить собственную команду, которую сервер без должной проверки передаёт в командную строку, последствия могут быть катастрофическими: от получения доступа к конфиденциальным данным до полного захвата контроля над сервером.

Как работает Command Execution?

Уязвимость command execution возникает, когда веб-приложение не проверяет входные данные должным образом. Злоумышленник может отправить специально сформированный запрос, который будет интерпретирован как команда операционной системы. В результате сервер выполнит эту команду, что может привести к нежелательным последствиям для системного администратора, таким как несанкционированный доступ к серверу и полный контроль над ним.
Уязвимое веб-приложение будет использоваться как интерфейс к операционной системе.

Приведем пример уязвимого PHP кода, который может позволить запустить любые команды, преданные от пользователя:

```
if (isset($_POST["command"])){
  echo "<p class='output'>" . shell_exec("find " . $_POST["command"]) . "
</p>";
}
```

Небольшой комментарий к коду:

- `isset($_POST["command"])` проверяет наличие переданного параметра `command` в POST-запросе;
- `shell_exec()` выполняет переданную строку как системную команду и возвращает результат;
- Отсутствие валидации и фильтрации пользовательского ввода позволяет злоумышленнику подставить любые команды, в том числе потенциально опасные

(например, `; rm -rf /` или `&& cat /etc/passwd`).

> **Важно:** Использование `shell_exec()` без строгой фильтрации входных данных представляет критическую угрозу безопасности.

Уязвимость File Upload

File Upload — это тип уязвимости, связанный с недостаточной проверкой загружаемых пользователем файлов. Если веб-приложение позволяет загрузку файлов без должного контроля, злоумышленник может использовать это для выполнения различных атак.

Основные векторы атаки:

- **Загрузка веб-оболочки или вредоносного скрипта.** В случае успеха злоумышленник получает возможность удалённого управления сервером через интерфейс веб-приложения.
- **Получение конфиденциальной информации.** Через неправильно настроенные загрузчики файлов можно получить доступ, например, к конфигурационным файлам, содержащим логины и пароли к базе данных.

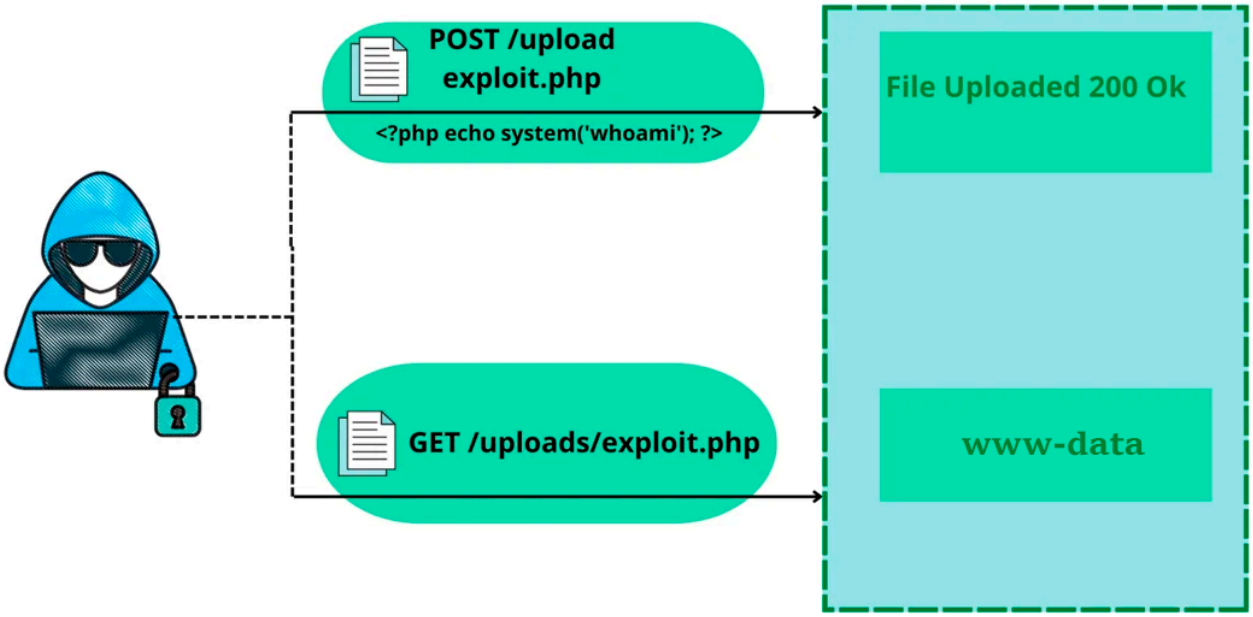


Рис. 37 Демонстрация загрузки вредоносного файла с целью удалённого выполнения кода

Например, можно зачитать файл, содержащий информацию о подключении к БД, тем самым узнав пароль и логин пользователя, который имеет к ней доступ. Данная информация может позволить получить доступ к серверу вею-приложения..

Приведем пример уязвимого кода, который позволяет загрузить любой файл на удаленный сервер:

```
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file" id="file"><br>
    <input type="submit" name="submit" value="Подтвердить">
</form>

<?php
    if(isset($_POST["submit"])) {
        move_uploaded_file($_FILES["file"]["tmp_name"], "uploads/" .
basename($_FILES["file"]["name"]));
    }
?>
```

- `<form>...</form>` - данный фрагмент кода представляет собой HTML-форму, которая позволяет пользователю загрузить файл на сервер. Данные будут отправлены с использованием метода POST и типом содержимого `multipart/form-data` (позволяет отправлять несколько частей данных (файлы) в одном запросе, каждая из которых имеет свой заголовок и тело).
- `<?php ... ?>` - PHP-скрипт, который проверяет существование элемента формы с именем "submit", используя встроенную функцию `isset`.
- Встроенная функция `move_uploaded_file()` перемещает файл из временного хранилища в указанное место. Первый аргумент функции — временное имя файла, а второй — путь к папке назначения с именем файла.
- `basename()` возвращает имя файла без пути к нему. Таким образом, код перемещает загруженный файл во вложенную папку `uploads` с сохранением оригинального имени файла.

Так как данные не проходят никакую обработку, пользователь может загрузить любой файл с любым расширением и полностью раскрыть уязвимость File Upload.

Потенциальные последствия

1. **Удалённое выполнение кода:** злоумышленник может загрузить файл с расширением `.php`, содержащий вредоносный скрипт, и запустить его, получив удалённый доступ к серверу.
2. **Доступ к чувствительным данным:** с помощью специально сконструированного файла можно считать содержимое конфигурационных файлов (например, `config.php`), где часто хранятся логины и пароли.
3. **Расширение привилегий:** успешная загрузка может стать первым шагом для дальнейшего распространения по системе.